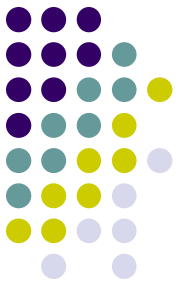


Number Representation

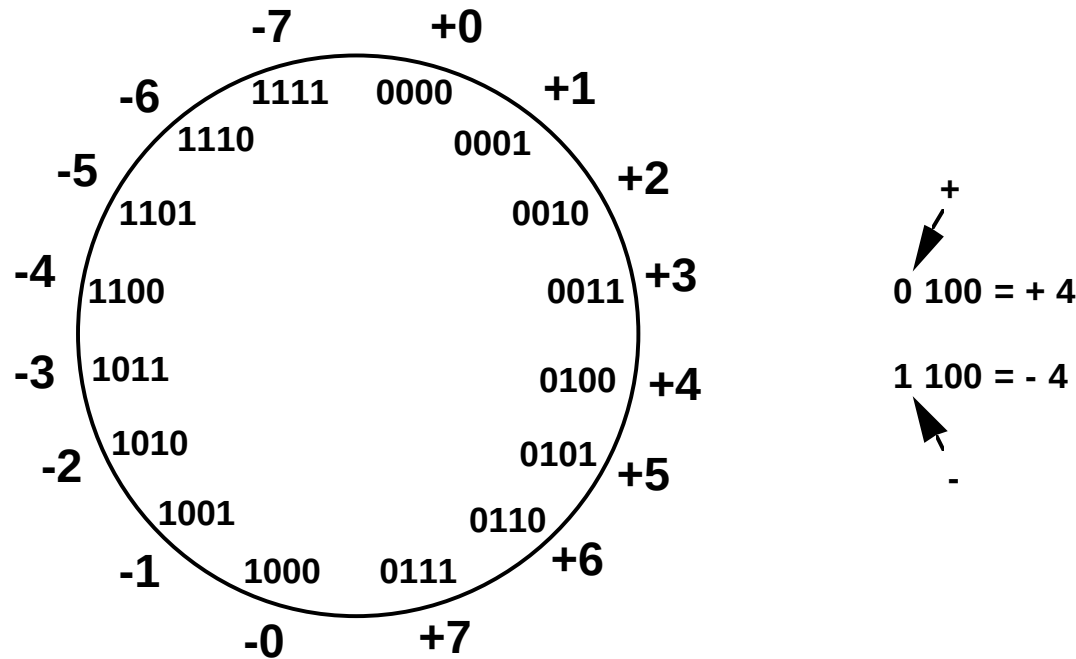
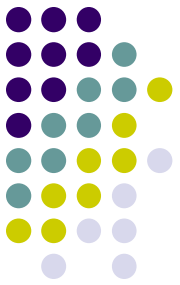
- Number System Representations.
- Addition of Unsigned Integers.
- Addition and Subtraction of Signed Integers.
- Addition/subtraction in the 2's-complement system.



Signed Integer

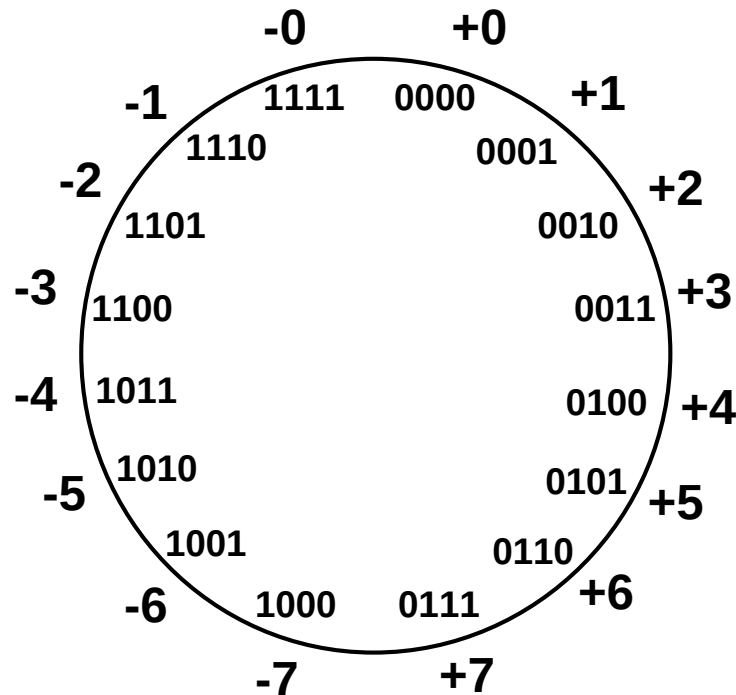
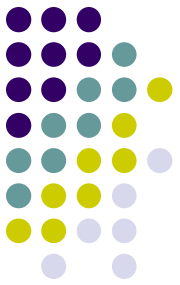
- 3 major representations:
 - Sign and magnitude
 - One's complement
 - Two's complement
- Assumptions:
 - 4-bit machine word
 - 16 different values can be represented
 - Roughly half are positive, half are negative

Sign and Magnitude Representation



High order bit is sign: 0 = positive (or zero), 1 = negative
Three low order bits is the magnitude: 0 (000) thru 7 (111)
Number range for n bits = $\pm 2^{n-1} - 1$
Two representations for 0

One's Complement Representation

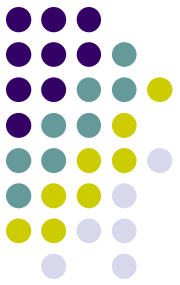


$\begin{array}{c} + \\ \swarrow \\ 0\ 100 = +4 \end{array}$

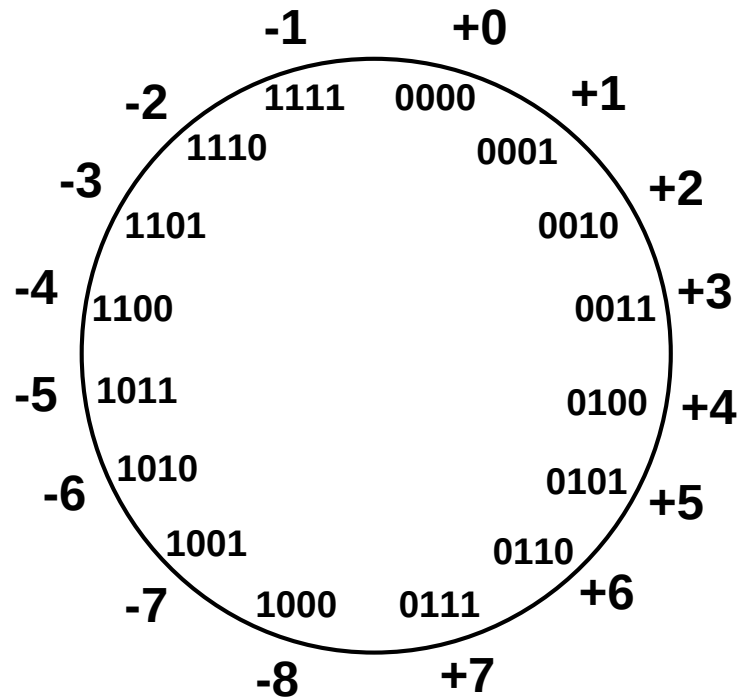
$\begin{array}{c} 1\ 011 = -4 \\ \searrow \\ - \end{array}$

- Subtraction implemented by addition & 1's complement
- Still two representations of 0! This causes some problems
- Some complexities in addition

Two's Complement Representation



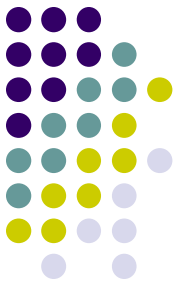
*like 1's comp
except shifted
one position
clockwise*



$0 \text{ } 100 = +4$
 $1 \text{ } 100 = -4$

- Only one representation for 0
- One more negative number than positive number

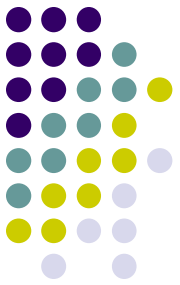
Binary, Signed-Integer Representations



Page 28

B $b_3 b_2 b_1 b_0$	Values represented		
	Sign and magnitude	1's complement	2's complement
0 1 1 1	+ 7	+ 7	+ 7
0 1 1 0	+ 6	+ 6	+ 6
0 1 0 1	+ 5	+ 5	+ 5
0 1 0 0	+ 4	+ 4	+ 4
0 0 1 1	+ 3	+ 3	+ 3
0 0 1 0	+ 2	+ 2	+ 2
0 0 0 1	+ 1	+ 1	+ 1
0 0 0 0	+ 0	+ 0	+ 0
1 0 0 0	- 0	- 7	- 8
1 0 0 1	- 1	- 6	- 7
1 0 1 0	- 2	- 5	- 6
1 0 1 1	- 3	- 4	- 5
1 1 0 0	- 4	- 3	- 4
1 1 0 1	- 5	- 2	- 3
1 1 1 0	- 6	- 1	- 2
1 1 1 1	- 7	- 0	- 1

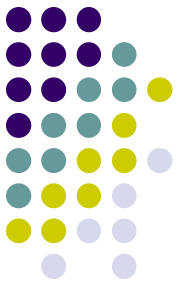
Figure 2.1. Binary, signed-integer representations.



Rules for Addition & Subtraction in 2's Complement system

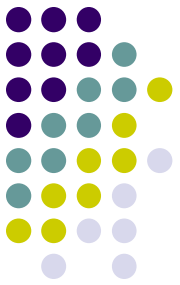
- To add two numbers: add their n -bit representations, ignoring the carry-out bit from the most significant bit (MSB) position. The sum will be the algebraically correct value in 2's-complement representation if the actual result is in the range -2^{n-1} through $+2^{n-1} - 1$.
- To subtract two numbers X and Y , that is, to perform $X - Y$, form the 2's-complement of Y , then add it to X using the add rule. Again, the result will be the algebraically correct value in 2's-complement representation if the actual result is in the range -2^{n-1} through $+2^{n-1} - 1$.

Why 2's complement is better?



- Add +5 and -3
- Add -4 and +2

Addition and Subtraction – 2's Complement



If carry-in to the high order bit = carry-out then ignore carry

$$\begin{array}{r}
 4 \quad 0100 \\
 + 3 \quad 0011 \\
 \hline
 7 \quad 0111
 \end{array}$$

$$\begin{array}{r}
 -4 \quad 1100 \\
 + (-3) \quad 1101 \\
 \hline
 -7 \quad 11001
 \end{array}$$

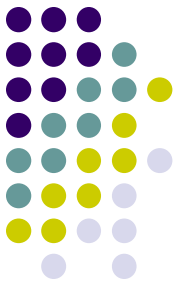
if carry-in differs from carry-out then overflow

$$\begin{array}{r}
 4 \quad 0100 \\
 - 3 \quad 1101 \\
 \hline
 1 \quad 10001
 \end{array}$$

$$\begin{array}{r}
 -4 \quad 1100 \\
 + 3 \quad 0011 \\
 \hline
 -1 \quad 1111
 \end{array}$$

Simpler addition scheme makes twos complement the most common choice for integer number systems within digital systems

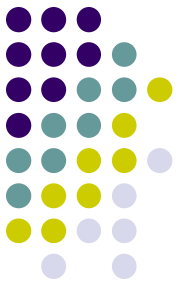
2's-Complement Add and Subtract Operations



Page 31

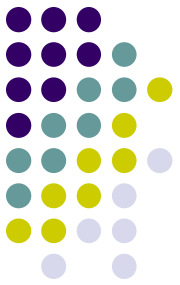
(a)	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{l} (+2) \\ (+3) \\ \hline (+5) \end{array}$	(b)	$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$	$\begin{array}{l} (+4) \\ (-6) \\ \hline (-2) \end{array}$
(c)	$\begin{array}{r} 1011 \\ + 1110 \\ \hline 1001 \end{array}$	$\begin{array}{l} (-5) \\ (-2) \\ \hline (-7) \end{array}$	(d)	$\begin{array}{r} 0111 \\ + 1101 \\ \hline 0100 \end{array}$	$\begin{array}{l} (+7) \\ (-3) \\ \hline (+4) \end{array}$
(e)	$\begin{array}{r} 1101 \\ - 1001 \\ \hline \end{array}$	$\begin{array}{l} (-3) \\ (-7) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1101 \\ + 0111 \\ \hline 0100 \end{array}$	$\begin{array}{l} \\ \\ \hline (+4) \end{array}$
(f)	$\begin{array}{r} 0010 \\ - 0100 \\ \hline \end{array}$	$\begin{array}{l} (+2) \\ (+4) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0010 \\ + 1100 \\ \hline 1110 \end{array}$	$\begin{array}{l} \\ \\ \hline (-2) \end{array}$
(g)	$\begin{array}{r} 0110 \\ - 0011 \\ \hline \end{array}$	$\begin{array}{l} (+6) \\ (+3) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0110 \\ + 1101 \\ \hline 0011 \end{array}$	$\begin{array}{l} \\ \\ \hline (+3) \end{array}$
(h)	$\begin{array}{r} 1001 \\ - 1011 \\ \hline \end{array}$	$\begin{array}{l} (-7) \\ (-5) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$	$\begin{array}{l} \\ \\ \hline (-2) \end{array}$
(i)	$\begin{array}{r} 1001 \\ - 0001 \\ \hline \end{array}$	$\begin{array}{l} (-7) \\ (+1) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 1001 \\ + 1111 \\ \hline 1000 \end{array}$	$\begin{array}{l} \\ \\ \hline (-8) \end{array}$
(j)	$\begin{array}{r} 0010 \\ - 1101 \\ \hline \end{array}$	$\begin{array}{l} (+2) \\ (-3) \\ \hline \end{array}$	$\Rightarrow$	$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array}$	$\begin{array}{l} \\ \\ \hline (+5) \end{array}$

Figure 2.4. 2's-complement Add and Subtract operations.



Overflow in Integer Arithmetic

- In unsigned numbers, carry out results an overflow
- In signed numbers, carry-out bit from the sign-bit position is not an indication of overflow.
- Overflow may occur only if both summands have the same sign.
- Add +7 and +4
- Add -4 and -6



Overflow Conditions

$$\begin{array}{r} 5 \qquad 0111 \\ \quad \quad 0101 \\ \hline 3 \qquad 0011 \\ \hline -8 \qquad 1000 \end{array}$$

Overflow

$$\begin{array}{r} 5 \qquad 0000 \\ \quad \quad 0101 \\ \hline 2 \qquad 0010 \\ \hline 7 \qquad 0111 \end{array}$$

No overflow

$$\begin{array}{r} -7 \qquad 1000 \\ \quad \quad 1001 \\ \hline -2 \qquad 1100 \\ \hline 7 \qquad 10111 \end{array}$$

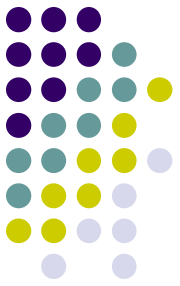
Overflow

$$\begin{array}{r} -3 \qquad 1111 \\ \quad \quad 1101 \\ \hline -5 \qquad 1011 \\ \hline -8 \qquad 11000 \end{array}$$

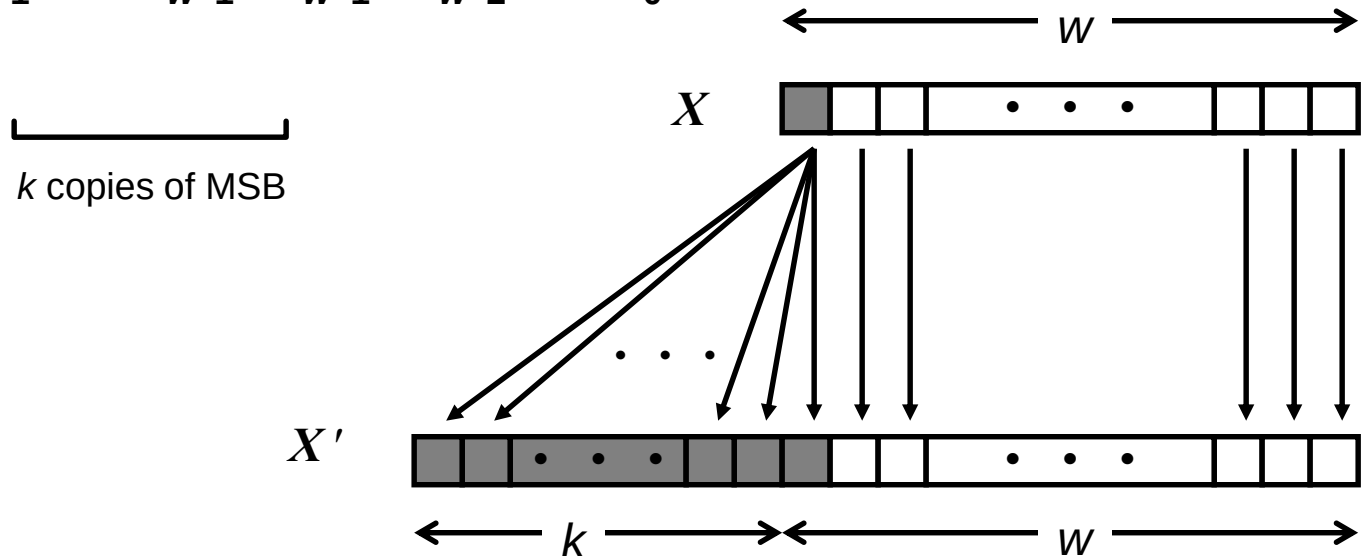
No overflow

Overflow when carry-in to the high-order bit does not equal carry out

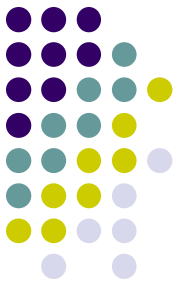
Sign Extension



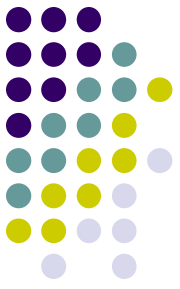
- Task:
 - Given w -bit signed integer x
 - Convert it to $w+k$ -bit integer with same value
- Rule:
 - Make k copies of sign bit:
 - $X' = x_{w-1}, \dots, x_{w-1}, x_{w-1}, x_{w-2}, \dots, x_0$



Memory Locations, Addresses, and Operations



Memory Location, Addresses, and Operation



- Memory consists of many millions of storage cells, each of which can store 1 bit.
- Data is usually accessed in n -bit groups. n is called word length.

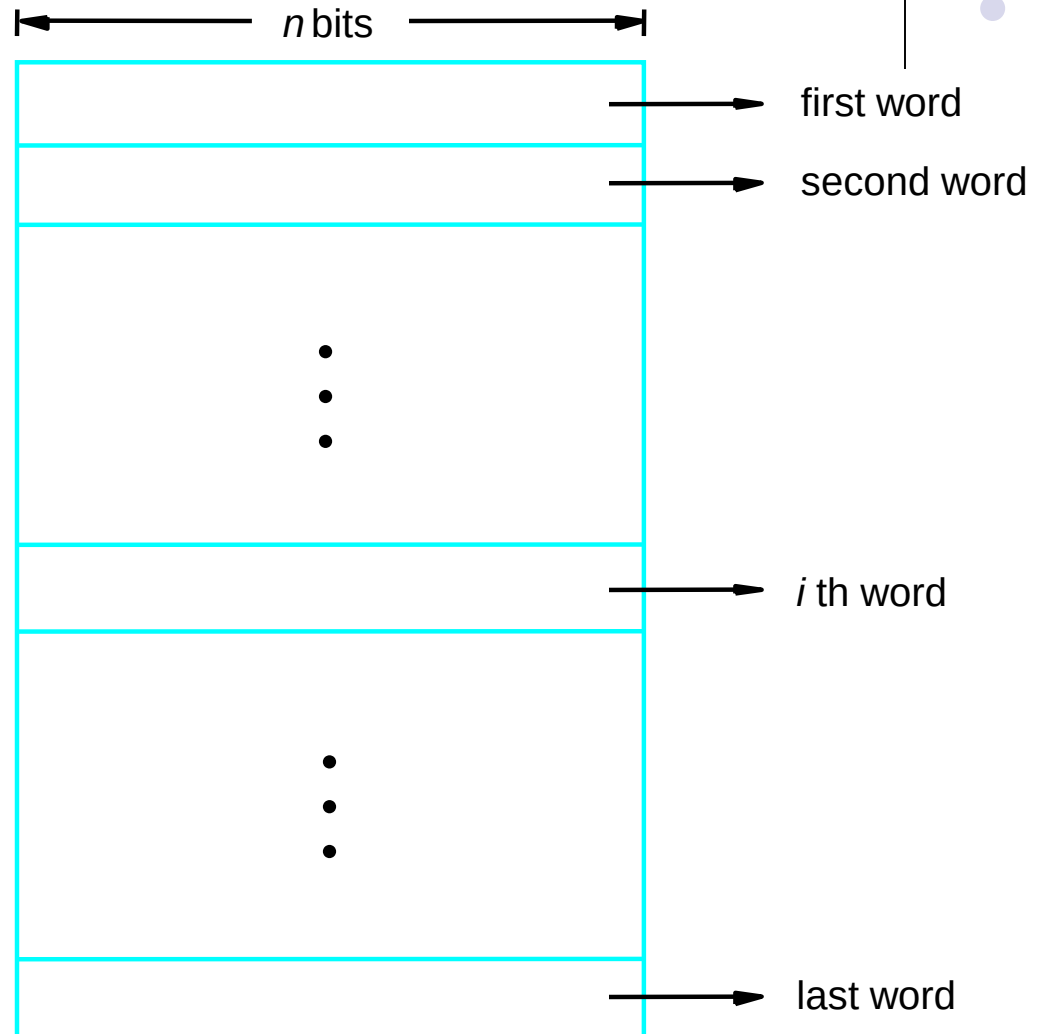
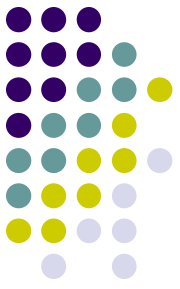
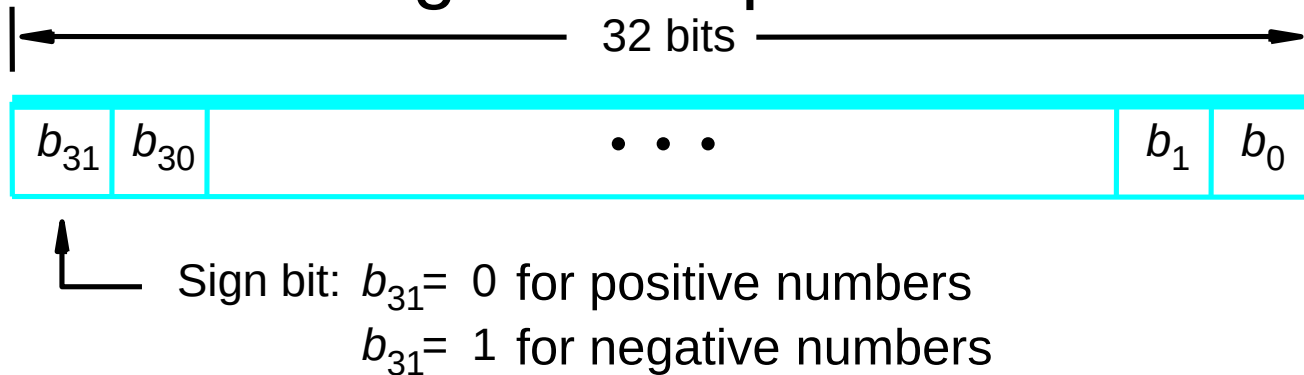


Figure 2.5. Memory words.

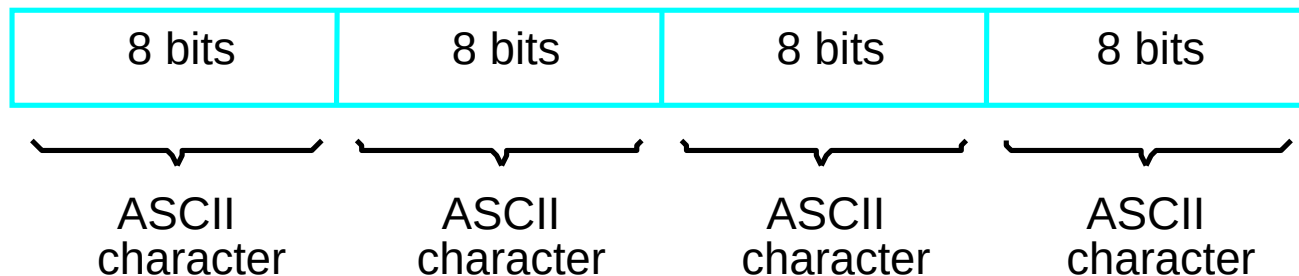
Memory Location, Addresses, and Operation



- 32-bit word length example

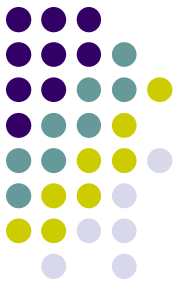


(a) A signed integer



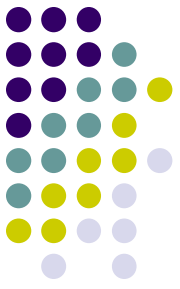
(b) Four characters

Memory Location, Addresses, and Operation



- To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.
- A k -bit address memory has 2^k memory locations, namely $0 - 2^k - 1$, called memory space.
- 24-bit memory: $2^{24} = 16,777,216 = 16\text{M}$ ($1\text{M} = 2^{20}$)
- 32-bit memory: $2^{32} = 4\text{G}$ ($1\text{G} = 2^{30}$)
- $1\text{K}(\text{kilo}) = 2^{10}$
- $1\text{T}(\text{tera}) = 2^{40}$

Memory Location, Addresses, and Operation



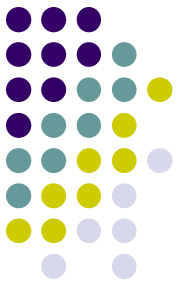
- It is impractical to assign distinct addresses to individual bit locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory – byte-addressable memory.
- Byte locations have addresses 0, 1, 2, ... If word length is 32 bits, they successive words are located at addresses 0, 4, 8,...

Little-Endian: opposite ordering. lower byte addresses are used for the less significant bytes of the word



Figure 2.7. Byte and word addressing.

Memory Location, Addresses, and Operation



- Address ordering of bytes
- Word alignment
 - Words are said to be aligned in memory if they begin at a byte addr. that is a multiple of the num of bytes in a word.
 - 16-bit word: word addresses: 0, 2, 4,....
 - 32-bit word: word addresses: 0, 4, 8,....
 - 64-bit word: word addresses: 0, 8, 16,....
- Access numbers, characters, and character strings